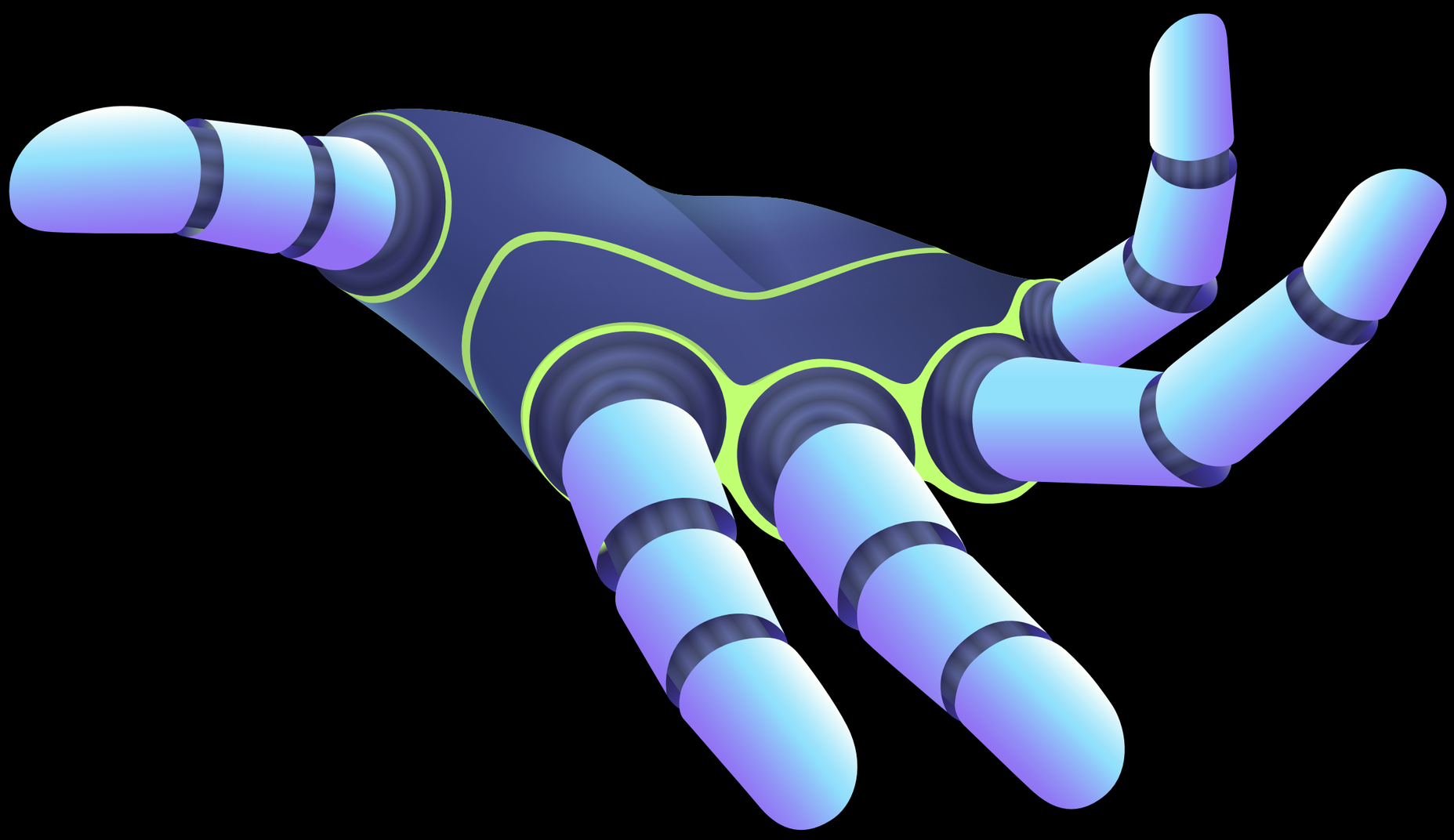
RAG

From Concept to Implementation



bipminds_tech

Naive RAG

User Query

A query or question is given by the user as input.

Embedding

The query is converted into vector embeddings (numerical representations) using an embedding model.

These embeddings help in semantic search and comparison with data sources.

Vector Database (Vector DB)

The embeddings are compared against a vector database, which contains pre-embedded knowledge from data sources. The most relevant results are retrieved.

Prompt Augmentation

Retrieved results are then combined with the original user query.

This augmented prompt gives more context to the generative model.

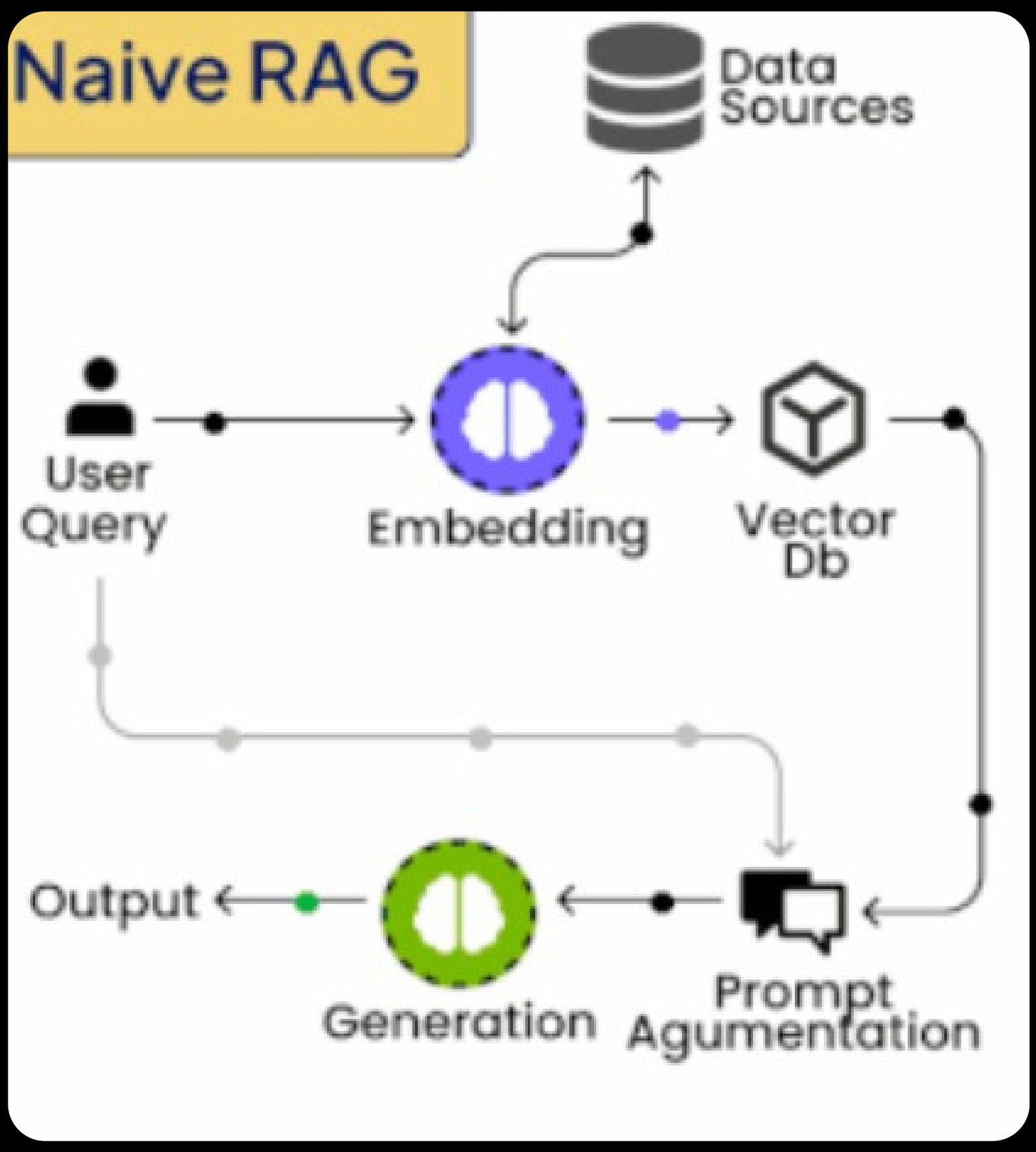
Generation

A large language model (LLM) uses the augmented prompt to generate a final response.

Output

The system returns the generated output to the user.

User submits a **query.**
Query is **embedded** into vector form.
Embeddings are used to search in a **vector DB** (built from data sources).
Retrieved documents are fed into **prompt augmentation** (combined with query).
LLM performs **generation** to create a meaningful answer.
The **output** is returned to the user.



bipminds_tech

Graph RAG

User Query

The user provides a question or query to the system.

Embedding

The query is converted into vector embeddings (numerical format) that capture its semantic meaning.

Vector Database (Vector DB)

Like in Naive RAG, embeddings are used to retrieve relevant data chunks from a vector database built from data sources.

Graph Generator

This component analyzes retrieved data and constructs a knowledge graph, where entities (nodes) and their relationships (edges) are mapped. Helps in understanding structured connections between concepts.

Graph Database (Graph DB)

Stores the knowledge graph created by the Graph Generator.

Enhances retrieval by allowing queries not just on semantic similarity but also on relationships and context.

Prompt Augmentation

The retrieved results (from vector DB + graph DB) are combined with the user query.

This creates a richer prompt for the generative model with contextual and relational knowledge.

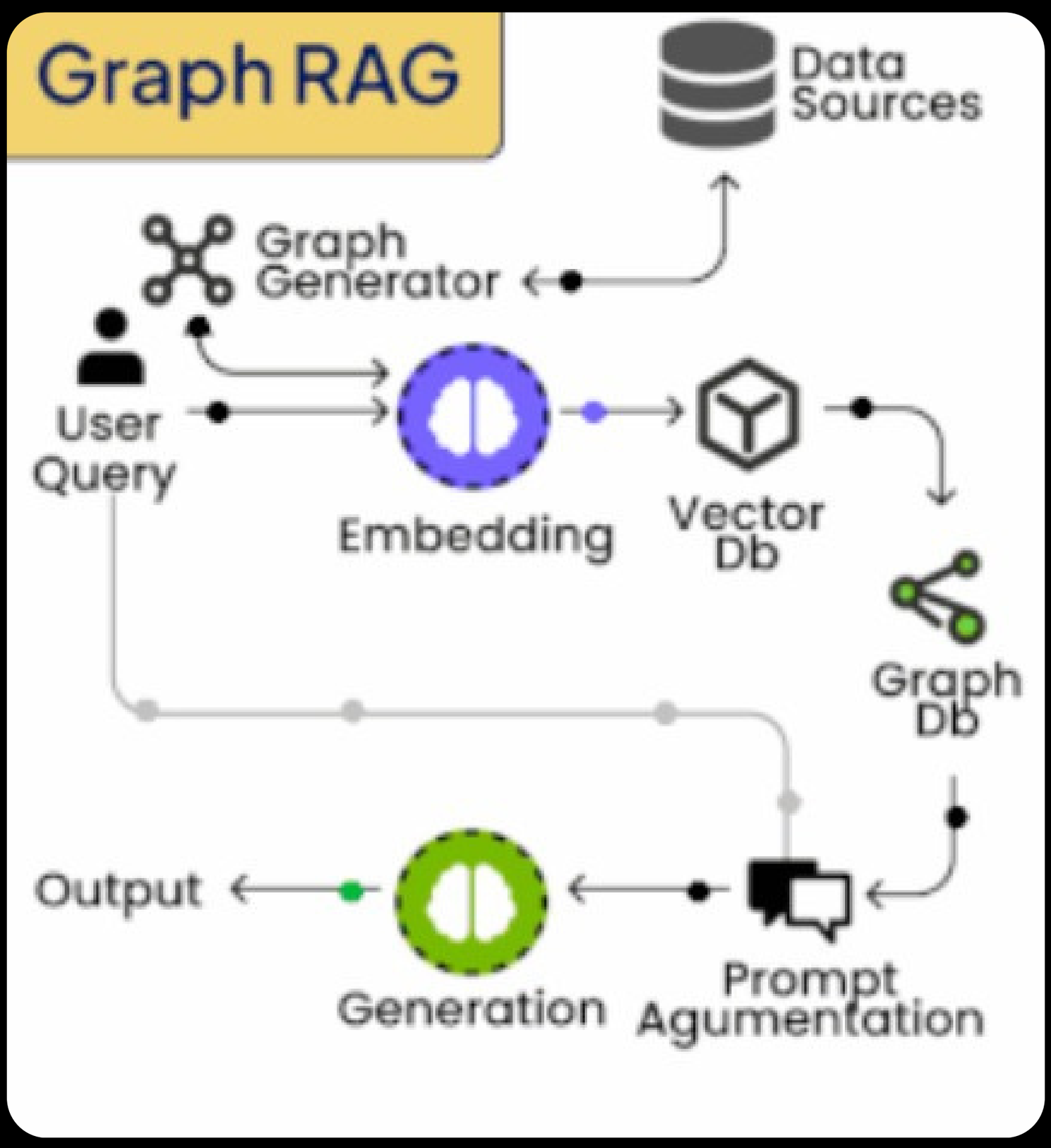
Generation

A large language model uses the augmented prompt to generate a detailed and accurate response.

Output

The final answer is returned to the user.

User enters a **query.**
The query is converted into **embeddings.**
Results are retrieved from the **vector DB.**
A **Graph Generator** structures these results into a **knowledge graph**, stored in a **graph DB.**
Both vector retrieval + graph-based knowledge are merged in **prompt augmentation.**
The **LLM** generates a response using this enriched input.
Output is returned to the user.



bipminds_tech

Hybrid RAG

User Query

A user provides a question or query.

This is the starting point of the system.

Embedding

The query is converted into embeddings (numerical vector representations).

These embeddings are used for semantic search and relationship mapping.

Graph Generator

Creates a knowledge graph from the data sources.

Captures entities (nodes) and their relationships (edges).

Provides a structured context for reasoning.

Vector Database (Vector DB)

Stores embeddings of textual data.

Provides Context 1: information retrieved based on semantic similarity to the user query.

Graph Database (Graph DB)

Stores knowledge graphs built from the Graph Generator.

Provides Context 2: information retrieved based on relational structures (entity-relationship context).

Prompt Augmentation

Merges Context 1 (Vector DB results) and Context 2 (Graph DB results) with the original query.

Creates a more comprehensive and context-rich prompt.

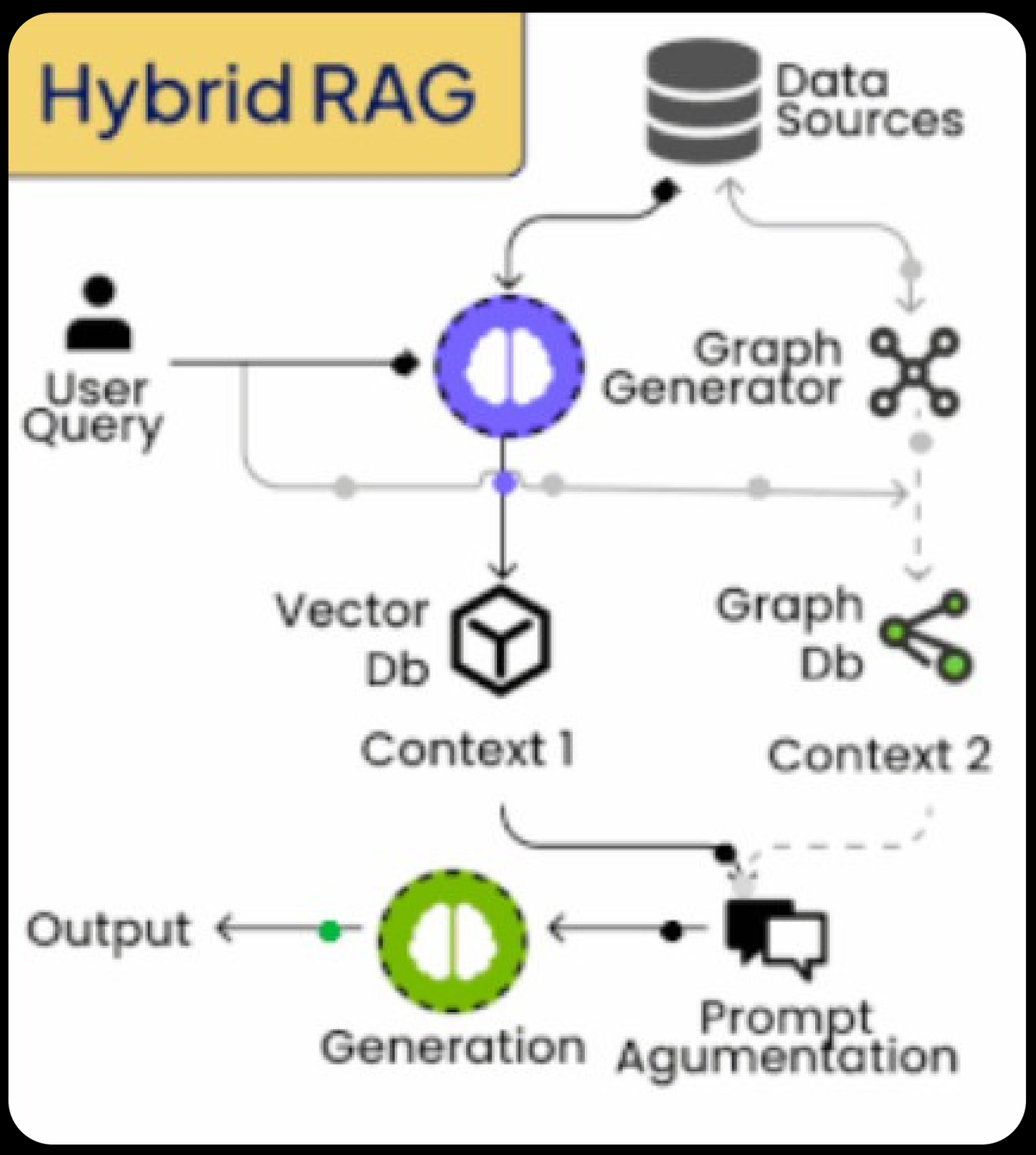
Generation

A large language model (LLM) generates the final response based on the enriched prompt.

Output

The generated answer is returned to the user.

User inputs a **query.**
Query is **embedded** into vectors.
Data sources feed into both **Vector DB** and **Graph Generator.**
Vector DB retrieval provides semantic similarity results (Context 1).
Graph DB retrieval provides structured relationship-based results (Context 2).
Both contexts are merged in **Prompt Augmentation.**
The **LLM** uses this combined knowledge to generate an accurate and contextually aware **output.**



HyDe

User Query

The user provides a query (question or request).

Hypothetical Document

Instead of directly embedding the query, the system first generates a hypothetical answer/document that could answer the query.

This document acts as a proxy, capturing more semantic detail than the original query alone.

Embedding

The hypothetical document is converted into embeddings.

These embeddings represent the query context more richly than just embedding the raw user query.

Vector Database (Vector DB)

Stores embeddings of real documents from the data sources.

The hypothetical document embeddings are used to retrieve the most semantically similar documents.

Prompt Augmentation

The retrieved results are combined with the user query (and sometimes the hypothetical doc).

This creates a more informative prompt for the language model.

Guide (Optional Flow)

The hypothetical document can also serve as guidance for the LLM during generation.

It helps align the generation with the intended meaning of the query.

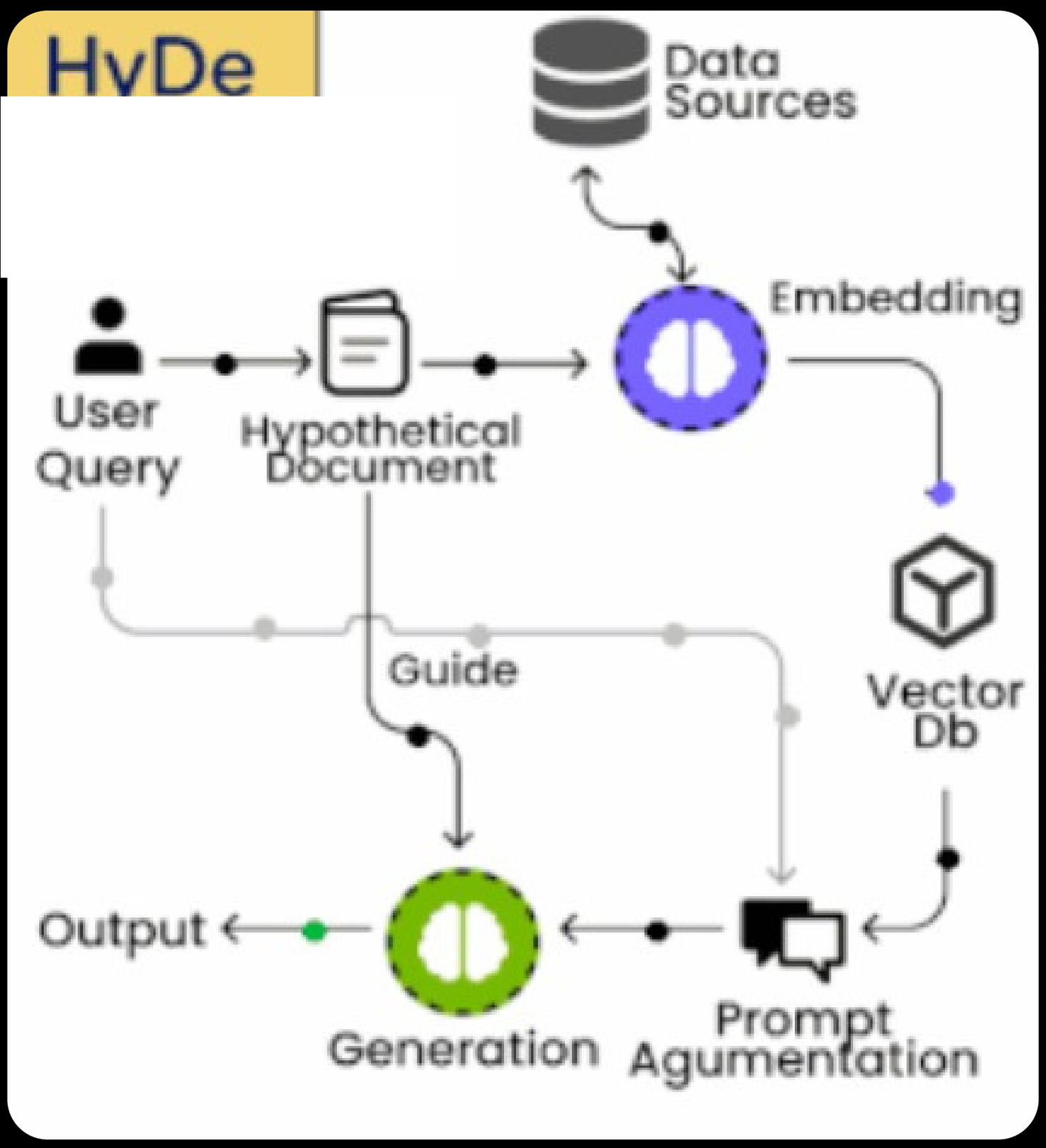
Generation

The LLM generates a final answer based on the augmented prompt and guidance.

Output

The system returns the answer to the user.

User enters a **query.**
The system generates a **hypothetical document** (like a draft answer).
This doc is **embedded** into vectors.
The embeddings are used to search in the **vector DB** for relevant real documents.
Retrieved documents + user query (and sometimes hypothetical doc) are merged in **prompt augmentation.**
The **LLM** generates the final output, optionally guided by the hypothetical document.
Answer is provided to the user.



bipminds_tech

Corrective RAG

User Query

The process begins with a user-provided query.

Embedding

The query is converted into embeddings, which represent its semantic meaning.

Grade (Evaluation Step)

Retrieved results are graded or validated.

The system checks if the retrieved documents are relevant and accurate.

Decision Point (Diamond Shape)

If the retrieved data is irrelevant or low quality (X mark) → correction is triggered.

If the data is relevant (✓ mark) → it continues as normal.

Search Web / Correct Info

If irrelevant results are detected, the system searches alternative sources (e.g., the web or external APIs) to gather correct information.

Vector Database (Vector DB)

Stores embeddings from trusted data sources.

Corrected info is fed back into the vector DB for improved retrieval in the future.

Prompt Augmentation

The validated or corrected results are merged with the original query.

Creates a refined and reliable prompt for the LLM.

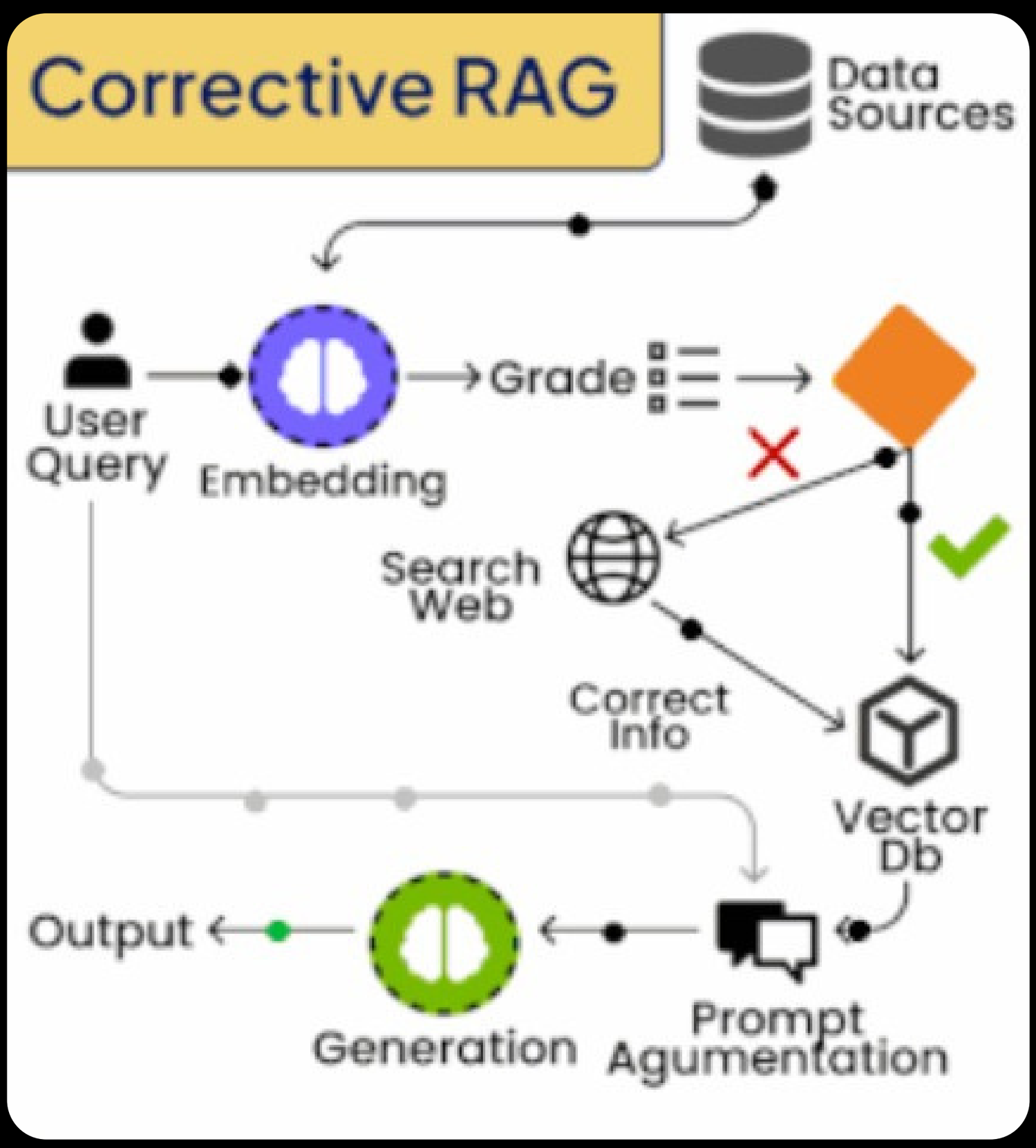
Generation

The LLM generates the final response using the corrected + augmented context.

Output

The refined answer is returned to the user.

User enters a **query.**
Query → **Embedding** → retrieve candidate documents.
Retrieved docs are **graded** for quality/relevance.
If correct → continue; if incorrect → **search web/correct sources**.
Corrected info is integrated into the **Vector DB.**
Final results go through **Prompt Augmentation.**
LLM generates a trustworthy output.
User receives a corrected and reliable answer.



bipminds_tech

Adaptive RAG

User

Provides a query or request as input.

Query Analyzer

Analyzes the user query to determine its complexity.

Decides whether the query can be handled directly, with a single retrieval step, or with multi-step reasoning.

Reasoning Chain

For complex queries, the system performs multi-step reasoning.

Breaks down the query into sub-questions and retrieves answers iteratively.

Helps in handling multi-hop reasoning, where the answer depends on connecting multiple pieces of evidence.

Embedding & Vector Database (Vector DB)

Embeddings are used to represent queries/documents in vector space.

The vector DB retrieves relevant information from the data sources.

Supports both single-step and multi-step retrieval.

Prompt Augmentation

The retrieved context (from direct, single, or multi-step paths) is combined with the query.

Produces a rich input prompt for the LLM.

Generation

The LLM generates the final answer based on the augmented prompt.

Output

The refined and context-aware answer is delivered to the user.

User submits a **query.**

Query Analyzer decides the path:

Direct → send query straight to the LLM.

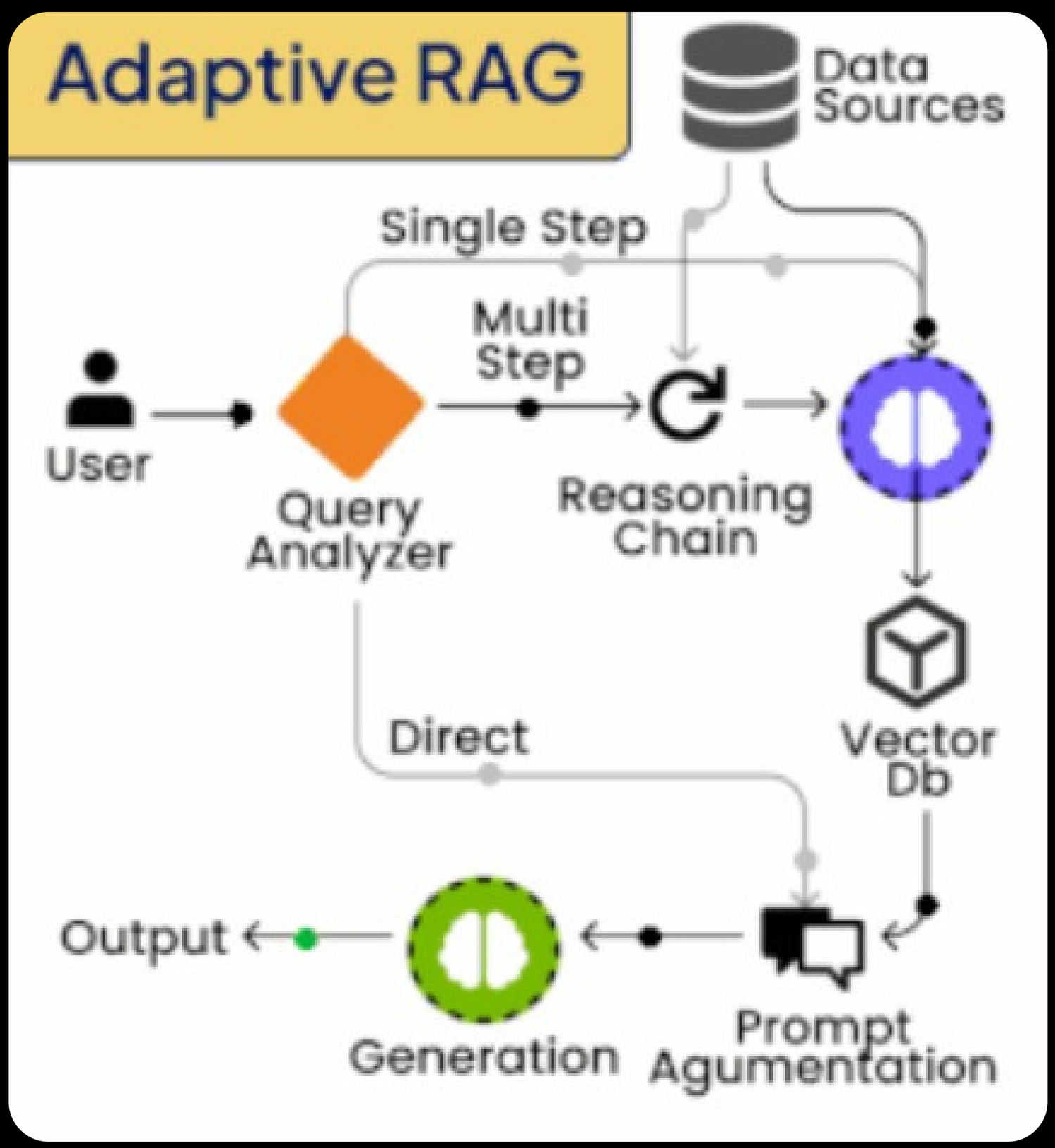
Single Step → one round of retrieval from Vector DB.

Multi Step → multi-hop reasoning via Reasoning Chain before retrieval.

Retrieved context is merged via **Prompt Augmentation.**

LLM Generation produces the answer.

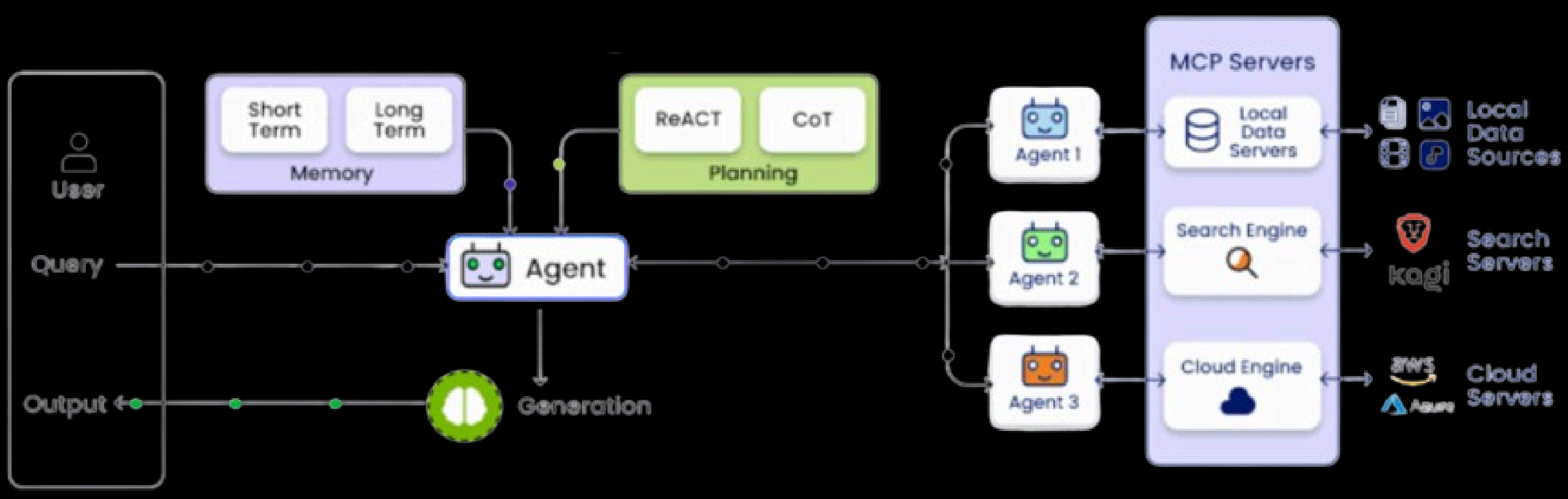
Final **output** is returned to the user.



bipminds_tech

Agentic RAG

The image depicts a multi-agent Retrieval-Augmented Generation (Agentic RAG) system where a central agent manages memory, planning, and generation, while delegating tasks to specialized agents connected to local data, search engines, and cloud servers. This setup makes the system more flexible, scalable, and intelligent in handling diverse queries.



bipminds_tech

bipminds_tech

follow for more

UPDATES